KV Forge: Mastering LLM Inference, from KV Cache to Production

Course syllabus · version 1 · approved 2026-10-21

Who it's for: Developers and platform engineers who run Python services and need to size, tune, operate and justify LLM serving; no GPU background assumed.

What you'll be able to do: Derive KV cache size and concurrency from a model config, tune an engine to a latency target, size replicas and cost per million tokens from QPS, run a serving service with routing, autoscaling and an eval gate, and defend the design in writing and in an interview.

40 lessons · about 15 hours

Tools you'll use

Tool	What for	Lessons	How you can run it
Python 3.12	the language for every lesson	all	installed with the course
vLLM	main serving engine	08, 11-17, 20-23, 27, 30-35	local (Docker, CPU image, no account) / GCP GPU VM (L4 spot, optional)
SGLang	second engine for comparison	15, 17	installed with the course
llama.cpp and Ollama	local serving comparison	15	installed with the course
LMCache	KV offload layer	13	installed with the course
LLM Compressor	weight and KV quantization	20, 21	installed with the course
vllm bench serve and GuideLLM	serving benchmarks	27	installed with the course
Locust	load generator	27, 33	installed with the course
Docker Compose	local multi-node cluster (mock engines, gateway, Prometheus)	08, 30, 33, 34, 35, 37	installed with the course
Prometheus client	metrics names and scraping	33	installed with the course
Google Compute Engine (GPU VM)	optional real GPU runs for G-labs	20, 21, 27	recorded results (no account) / GCE spot L4 (optional)

You'll be asked which option you want, and for any keys, just before the lessons that need it.

Before you start

• Docker installed (some lessons start services in containers)
• internet access to download models or datasets
• a GPU for a few lessons (optional: small CPU versions included)
Put the values in this course's .env file (template: .env.example).

Contents

Basic — 9 lessons · ~3 h

Chapter 1. Foundations and one server

01 Why LLM inference is its own problem · walkthrough — Explain why serving a model costs more and fails differently than running a web API, and map the course.

02 Decoding one token at a time · theory — Describe the autoregressive loop and why attention cost grows with context length.

03 KV cache size from a model config — Compute bytes per token from a config file and check it against Llama 3.1 8B (128 KiB per token in FP16) and 70B (320 KiB). Explain num_key_value_heads (GQA) in this lesson.

04 Attention variants: MHA, MQA, GQA and MLA · theory — Explain how the number of KV heads changes cache size and what each design gives up.

05 Max concurrency from the memory budget — Turn GPU memory into a concurrency limit: weights, activations and the KV budget.

06 Prefill, decode and the roofline — Predict TTFT and TPOT trends from arithmetic intensity and say which phase is compute-bound.

07 Measuring TTFT, TPOT and goodput — Measure time to first token, time per output token and goodput against an SLO.

08 Serve a model and call it: the API and streaming — Start an OpenAI-compatible server, call it with streaming, and keep API keys out of logs.

09 Checkpoint: capacity report for one model ★ — Put a concurrency estimate and a measured p95 TTFT against a target you set, on the CPU image or a recorded GPU result, for one model and one GPU.

Intermediate — 16 lessons · ~6 h

Chapter 2. Memory, batching and engines

10 Why paged memory: reserving the maximum wastes the GPU — Show, in a few lines of code, how reserving max length per request wastes memory and what that costs.

11 PagedAttention and the block allocator — Explain block tables and fragmentation, then build the allocator you would otherwise take on trust.

12 Continuous batching and chunked prefill — Explain iteration-level scheduling and the TTFT versus TPOT trade-off of chunked prefill.

13 Preemption and offload — Recognise when the scheduler preempts, why V1 recomputes instead of swapping, and where offload fits.

14 Prefix caching: hashes, radix trees and hit rate — Make a shared system prompt cacheable, measure the hit rate and find when caching stops helping.

15 Choosing a serving engine · walkthrough — Compare vLLM, SGLang, TensorRT-LLM, TGI and Triton, and where llama.cpp and Ollama fit for local serving.

16 The vLLM flags you will tune — Set memory, batch and length flags for a target, print the effective defaults from the startup log (gpu_memory_utilization 0.92, prefix caching and chunked prefill on), and justify each change.

17 Speculative decoding and multi-LoRA · theory — Explain draft-and-verify decoding and how adapters share one base model in memory.

18 Checkpoint: tune an engine to a TTFT target ★ — Tune one engine to a TTFT target and justify each flag with a measured number.

Chapter 3. Quantization, parallelism and hardware

19 Why parallelism: a 70B model does not fit one GPU — Count the weights and KV of a 70B model against one L4 and show why the model must be split.

20 Weight quantization: INT8, FP8, AWQ and GGUF — Choose a weight format for a memory target and measure what it saves.

21 KV cache quantization and measuring quality loss — Measure the accuracy delta of FP8 KV (vLLM documents FP8 and NVFP4 for KV; scales default to 1.0 without calibration) with a 50-prompt regression set, not perplexity alone.

22 Tensor parallelism and all-reduce · walkthrough — Explain how TP splits heads and MLPs and what the all-reduce per layer costs without NVLink.

23 Pipeline and expert parallelism; sizing a 70B model — Pick a TP degree and replica count for a 70B model and state the memory and communication cost.

24 GPU memory hierarchy and picking the cheapest GPU · theory — Compare L4, A100, H100 and H200 by memory, bandwidth and FP8 support, then pick the cheapest that fits, using a price you enter (GCP prices are not verified).

25 Checkpoint: pick a GPU and parallelism for an SLO ★ — Choose hardware, TP degree and replicas for a 70B SLO, with the arithmetic shown.

Advanced — 15 lessons · ~6 h

Chapter 4. Capacity and economics

26 Little's law: from QPS to concurrency — Turn a QPS target and latency into a concurrency number and a replica count with headroom.

27 Load tests that match reality — Drive a server with realistic length distributions and arrival rates, not fixed lengths.

28 Cost per million tokens and build vs buy ★ — Compute cost per million tokens at 20, 50 and 80 percent utilisation, show cache hit rate as the main cost lever, and compare on-prem, reserved, spot and on-demand breakeven.

Chapter 5. Serving at scale and operations

29 Why a gateway: one endpoint, many replicas — Build the failure in code: clients picking replicas themselves, measure the imbalance, then show what a gateway fixes.

30 Routing: prefix-aware and KV-aware balancing — Build the Compose mock vLLM server with real metric names, route across it by prefix and by token budget, and compare with round-robin.

31 Autoscaling on queue depth, not GPU utilisation — Scale replicas on the queue and KV signals, and account for cold-start delay.

32 Deploying the server: containers, config and GPU scheduling · walkthrough — Package the server, read its health checks and config, compare self-hosted, managed and serverless deployment, point to the vLLM Helm chart, KubeRay and LeaderWorkerSet, and validate manifests offline with kubeconform.

33 Reliability drill: OOM, node loss and the dashboard — Kill a replica of a Compose cluster of at least three nodes under load, read the vllm metrics, and print vllm:admission_rejections_total from /metrics on the pinned version.

34 Disaggregated prefill and multi-node serving · theory — Explain the prefill and decode split across nodes, and why the tensor-parallel rule is per node.

35 Rollouts with eval gates — Roll a new model out behind an eval gate and roll it back when quality drops.

36 Security and tenancy for an LLM endpoint · theory — Find the endpoints an API key does not protect, limit cross-tenant prefix-cache leaks with a per-tenant cache salt (vLLM does not isolate tenants), and keep prompts and PII out of logs with an audit trail.

37 Checkpoint: operate the mock cluster ★ — Run the Compose cluster of at least three nodes, kill one replica under load, and show the metrics that prove recovery.

Chapter 6. Enterprise and the capstone

38 Scale maths: 10x, 100x and HA/DR · theory — Extend a capacity model to 10x and 100x traffic, and name the RPO and RTO for each failure domain.

39 Platform patterns, frontier and case studies · theory — Explain SLOs and ownership for a shared platform, what changed in the last year, and three published write-ups.

40 Capstone: design the Llama 70B serving platform ★ — Write the 6-to-10-page design for 10,000 QPS across two regions and three tenants, from your own numbers.

★ = project lesson

Where next

Inference performance engineer · role · established

Makes model serving faster and cheaper per token, from engine internals down to kernels and profiling. Why next: Hiring is active now: the inference job sample of 95 postings from 27 companies includes performance-engineer roles.

- CUDA and Triton kernels
- FlashAttention
- NCCL collectives
- GPU profiling with Nsight
- C++ or Rust for hot paths

Start it: /tutorial "Inference performance engineering" basic reuse:kv-forge-15h

ML platform and inference SRE · role · established

Runs model serving as a service: SLOs, capacity, GPU fleets and incident response. Why next: The same job sample names SRE and serving-platform roles; reliability and capacity topics appear in 44% and 19% of postings.

- GPU fleet operations and DCGM
- SLO engineering
- capacity management
- multi-region failover

Start it: /tutorial "ML platform reliability" basic reuse:kv-forge-15h

Distributed inference on Kubernetes · technology · growing

Prefix-aware routing, prefill and decode disaggregation and tiered KV cache, as Kubernetes-native stacks. Why next: Several open projects now implement the routing and disaggregation ideas this course teaches as concepts.

- llm-d
- Gateway API Inference Extension
- KV offload tiers (LMCache)

Start it: /tutorial "Distributed inference on Kubernetes" basic reuse:kv-forge-15h

Appendix: details for reviewers

How this syllabus was checked, and other ways to take it. Learners can skip this part.
Running example — Brightside support desk assistant: every team has a support question to answer; a shared 1,500-token policy prompt makes prefix caching and KV budgets real one request (part 1) -> a tuned engine (part 2) -> a 70B model on several GPUs (part 3) -> capacity and cost for 10,000 QPS (part 4) -> replicas, routing and rollouts (part 5) -> a multi-region design (part 6)

Shorter paths through the same course

Path	Lessons	Time
Essentials	37	~14 h
Professional	38	~14 h
Complete	40	~15 h

Coverage report

42 concepts from 18 independent sources. Must-cover: 41; taught: 39; left out on purpose: 2.

Developer coverage (D1–D18)

Item	Lessons
D1 Purpose and architecture	01, 29
D2 Mental model and core concepts	02, 03
D3 Setup and first working app	08, 09
D4 Day-to-day API surface	08, 16
D5 Data, schemas and configuration	08, 32
D6 Integration with the app stack	30, 32
D7 Async, concurrency and performance basics	07, 27
D8 Errors, validation and reliability	33
D9 Testing and evaluation	09, 35
D10 Debugging and observability	07, 33
D11 Security	08, 36
D12 Packaging, deployment and operations	08, 32, 33
D13 Scaling and cost	28, 31
D14 Versions, upgrades and migration	15, 35
D15 Ecosystem and choosing	15, 28
D16 Common pitfalls and anti-patterns	13, 16, 33
D17 Reference architectures and real use cases	39, 40
D18 End-to-end capstone	40

Enterprise depth coverage

Item	Lessons
1 distributed architecture internals	34
2 cluster operations	30, 34
3 failure and recovery	33
4 extreme scale	38
5 deployment options	32
6 performance engineering at scale	27
7 high availability and disaster recovery	38

Item	Lessons
8 zero-downtime change	35
9 multi-tenancy and isolation	36
10 security and compliance at organisation scale	36
11 cost engineering	28
12 platform and organisation patterns	39
13 frontier	39
14 case studies	39

Out of scope

Training and fine-tuning models — research level — later, not serving
Terraform and infrastructure as code — the reference says no Terraform; lesson 32 names the pattern
Kubernetes labs — Kubernetes is its own course; lesson 32 teaches the concepts without a cluster

Syllabus history

Version	Date	Changes
1	2026-10-21	first approval